Data and Artificial Intelligence

Cyber Shujaa Program

Week 12 Assignment
Generative AI and Retrieval-Augmented Generation (RAG)

Student Name: Salome Kungu

Student ID: CS-DA03-26054

Contents

Data and Artificial Intelligence 1
Cyber Shujaa Program 1
Week 12 Assignment Generative AI and Retrieval-Augmented Generation (RAG) 1
Introduction 2
The key objectives of the assignment were to: 2
Tasks Completed 2
Task 1: Install Required Libraries and Import Libraries 2
Task 2: Define Text Cleaning Function 3
Task 3: Upload and Process the PDF 4
Task 4: Split Document into Chunks 5
Task 5: Create Embeddings and Vector Store 6
Task 6: Load the Generative Language Model 7
Task 7: Implement the RAG Query Function 8
Task 8: Execute the RAG Query 9
Conclusion 10

Introduction

This assignment focuses on implementing a **Retrieval-Augmented Generation (RAG) pipeline** to demonstrate how generative AI models can produce more accurate responses when supported by relevant document context.

The project involves processing a PDF document, splitting the document into smaller chunks, generating embeddings using **Sentence Transformers**, and storing these embeddings in a **FAISS vector database**. When a user submits a query, the system retrieves the most relevant document chunks and uses a **generative language model** to generate context-aware answers.

This approach improves the reliability and factual accuracy of responses because the model uses information retrieved directly from the document rather than relying only on general knowledge.

The key objectives of the assignment were to:

1. Apply generative AI techniques to synthesize answers from retrieved document content.
2. Extract text from a PDF document and split it into smaller chunks.
3. Generate embeddings using Sentence Transformers.
4. Store embeddings in a FAISS vector database for efficient retrieval.
5. Implement a Retrieval-Augmented Generation (RAG) pipeline using Python and Hugging Face models.

Tasks Completed

Link to Code via Google Colab:

https://colab.research.google.com/drive/1MOTl8SNaCW4MDOYrJzgMilbK6clFMfiF?usp=sharing

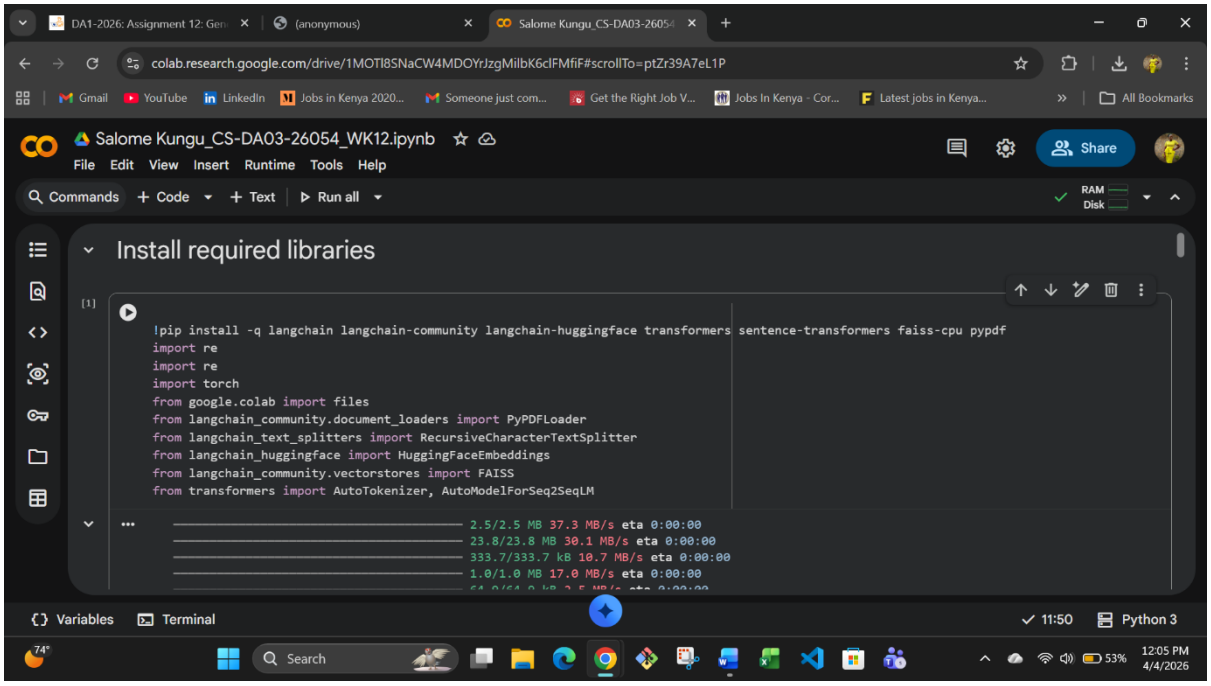
Task 1: Install Required Libraries and Import Libraries

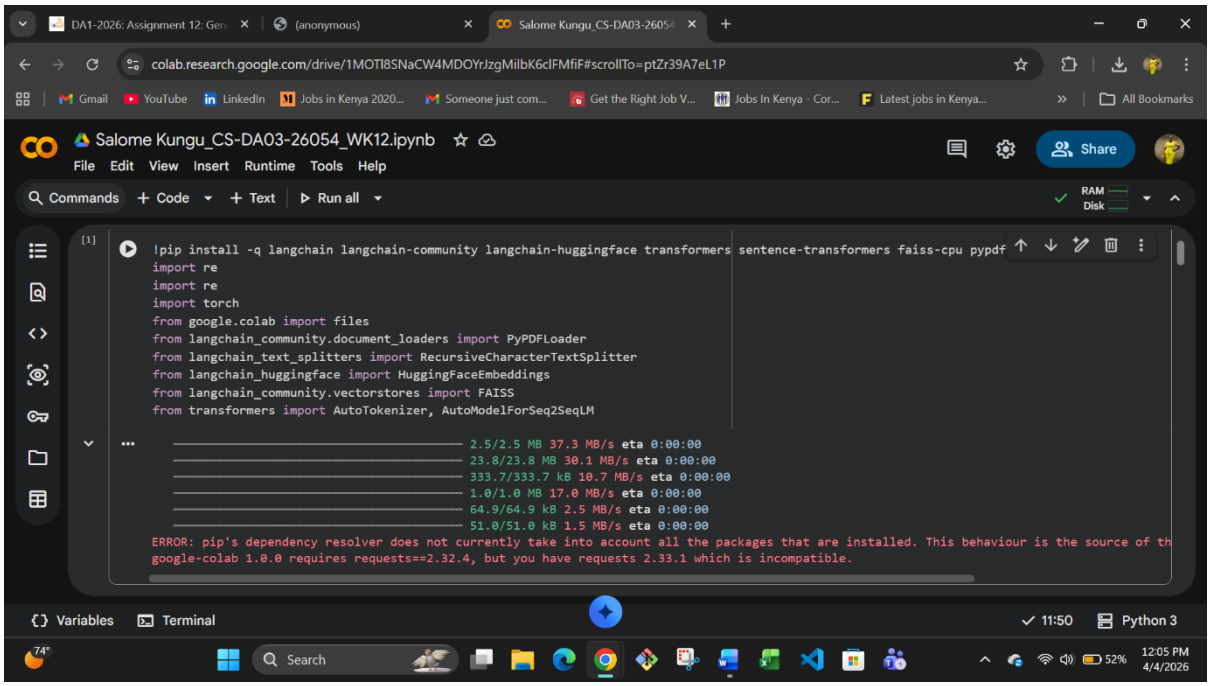
What

- Installed the necessary libraries including LangChain, Hugging Face Transformers, Sentence Transformers, FAISS, and PyPDF.
- Imported Python libraries required for document loading, text processing, embedding generation, and language model inference.

Why

- These libraries provide the tools required to implement the RAG pipeline. LangChain supports document processing and retrieval workflows, FAISS enables efficient vector similarity search, and Hugging Face models generate natural language responses.
- These libraries allow the program to process the PDF document, generate embeddings, perform vector search, and generate answers using a large language model.





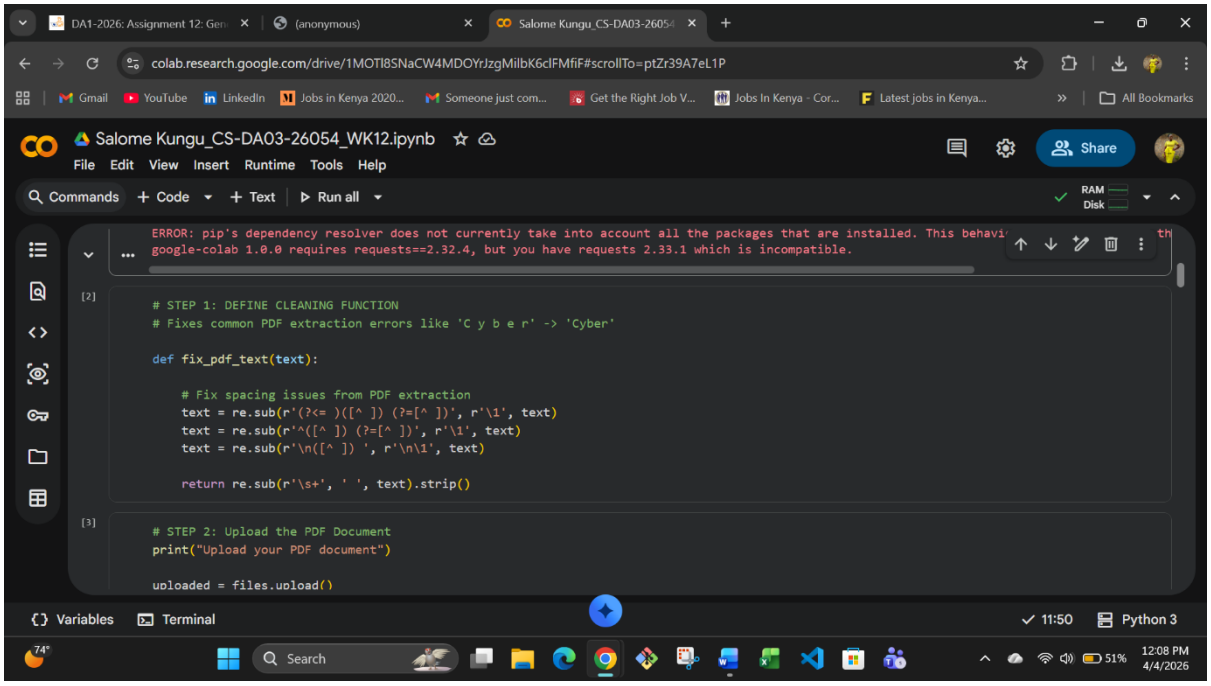
Task 2: Define Text Cleaning Function

What

- Created a function to clean extracted PDF text and fix formatting issues such as broken words and irregular spacing.

Why

- PDF extraction can introduce formatting errors. Cleaning the text improves the quality of embeddings and ensures the retrieval system works effectively.



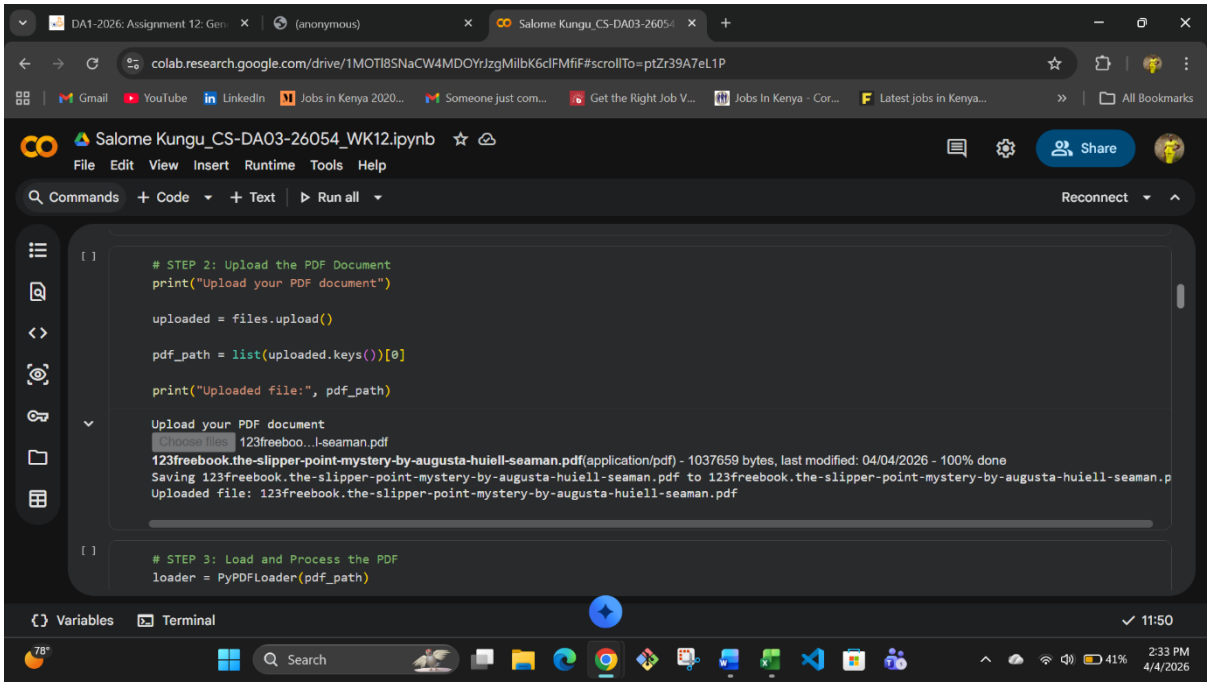
Task 3: Upload and Process the PDF

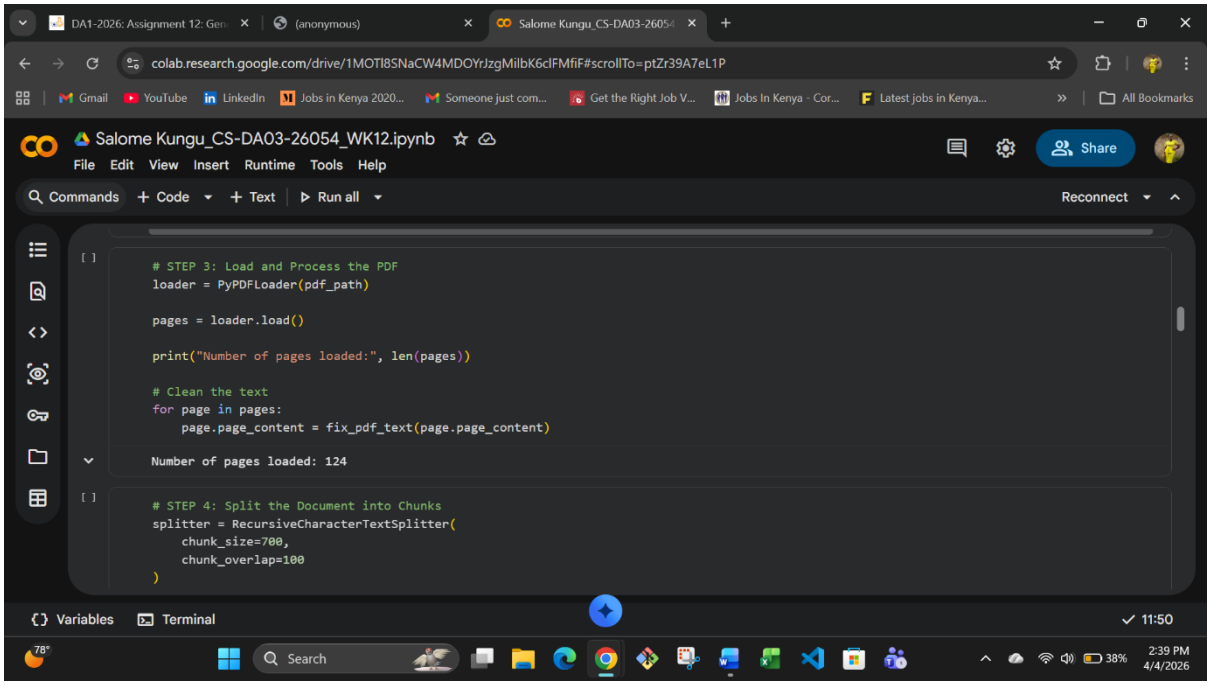
What

- Uploaded the PDF file and used **PyPDFLoader** to extract text from the document pages.

Why

- The PDF document serves as the **knowledge base** that the RAG system will use to answer user queries.





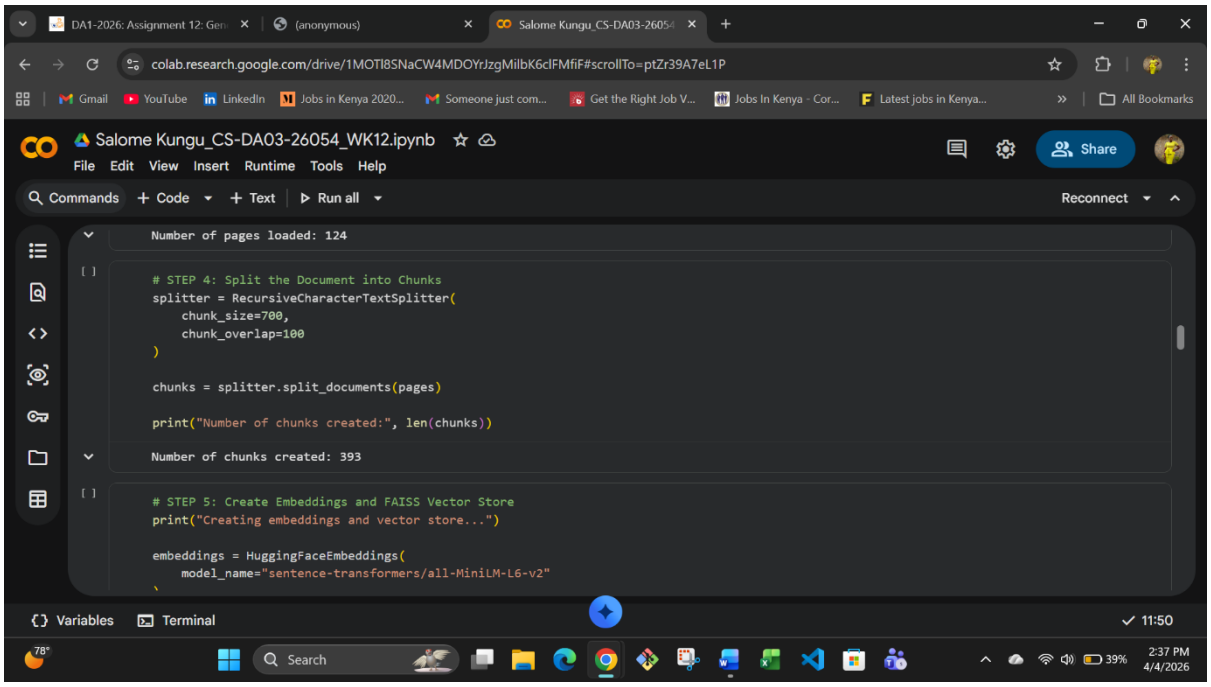
Task 4: Split Document into Chunks

What

- Used **RecursiveCharacterTextSplitter** to divide the document into smaller text chunks with overlapping sections.

Why

- Chunking allows the retrieval system to locate relevant parts of the document more efficiently and improves context retrieval for the language model.



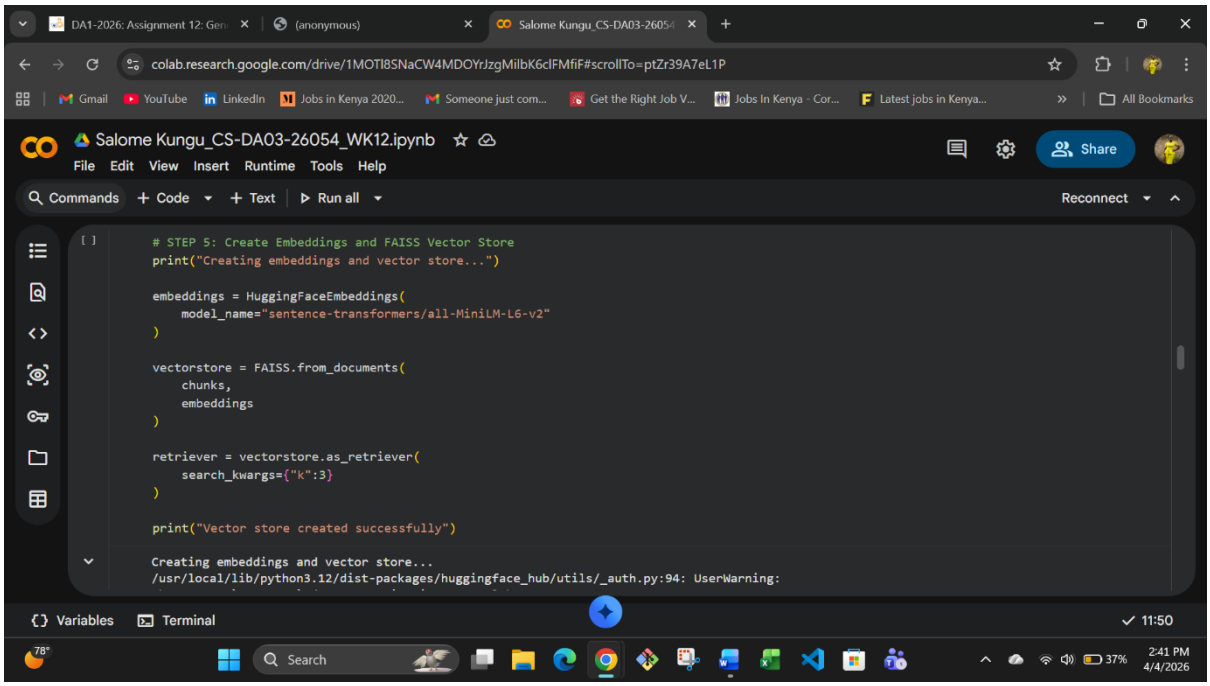
Task 5: Create Embeddings and Vector Store

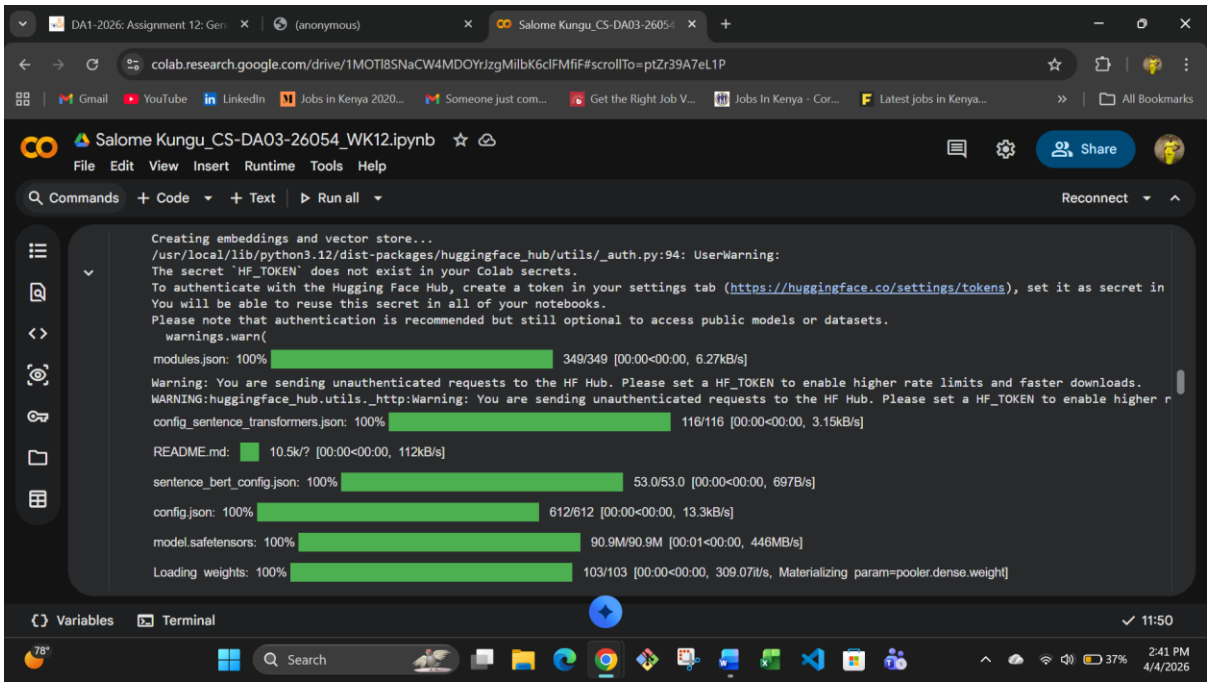
What

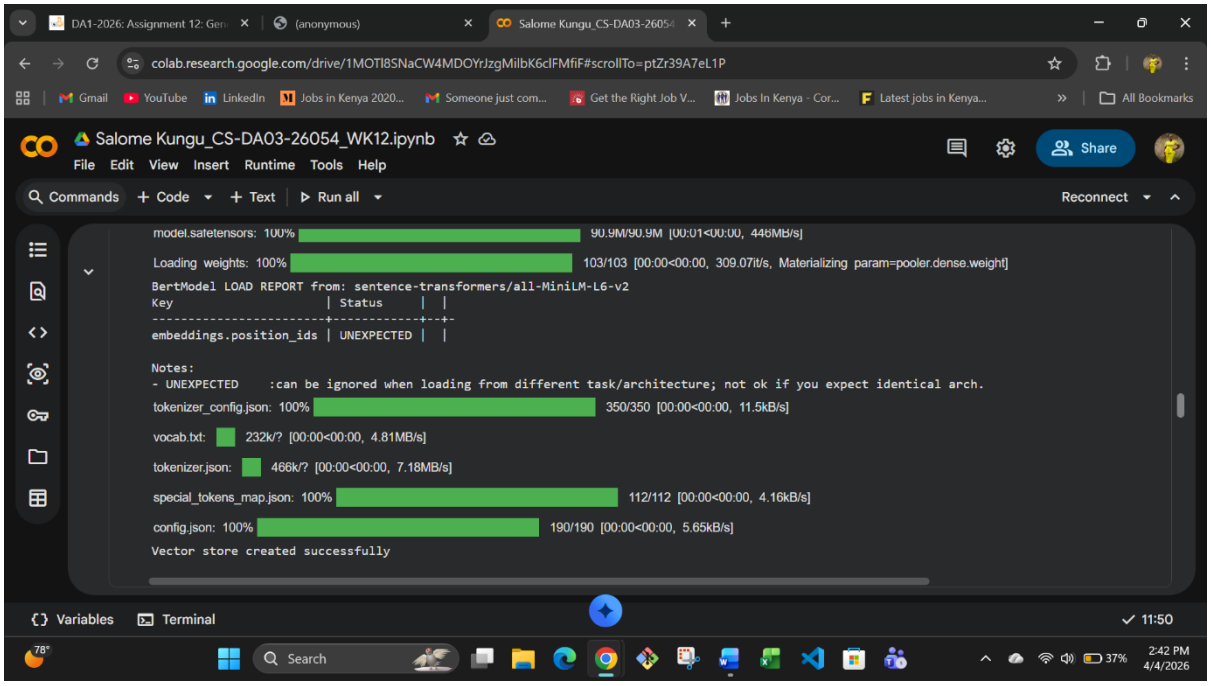
- Generated embeddings using the **Sentence Transformers model all-MiniLM-L6-v2** and stored them in a **FAISS vector database**.

Why

- Embeddings convert text into numerical vectors that capture semantic meaning. FAISS allows fast similarity search to retrieve the most relevant document chunks for a query







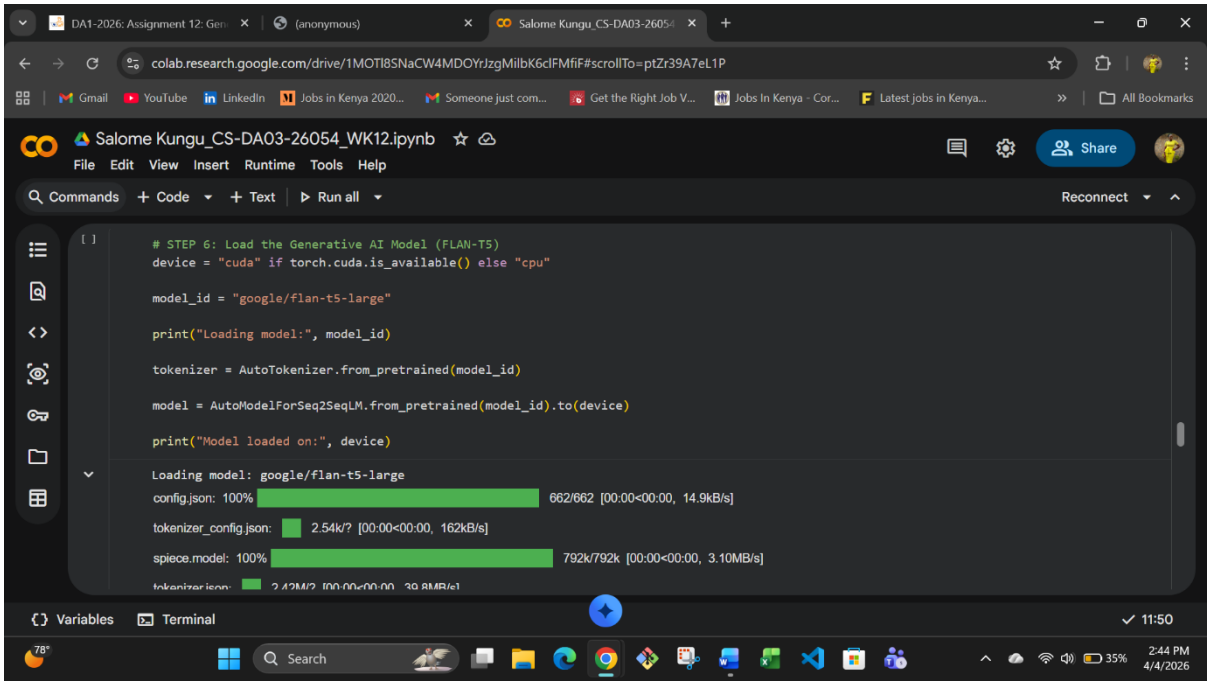
Task 6: Load the Generative Language Model

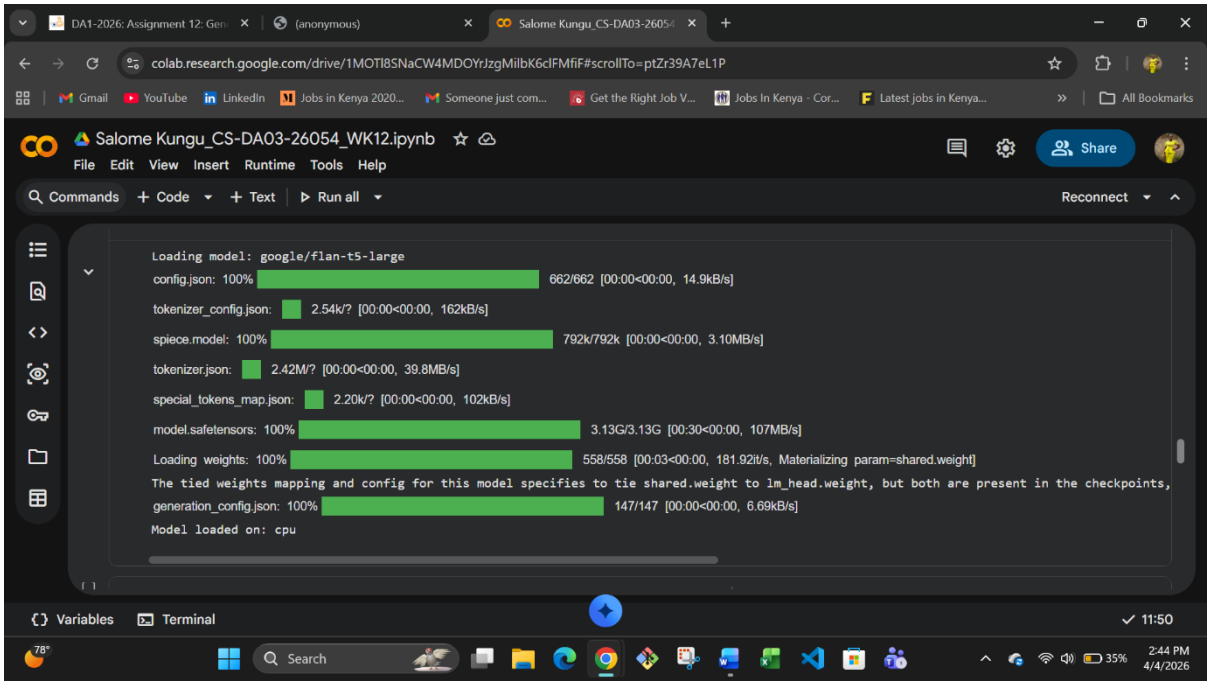
What

- Loaded the **FLAN-T5-Large** language model and tokenizer from Hugging Face.

Why

- The language model generates human-readable responses based on the retrieved document context and the user's query.





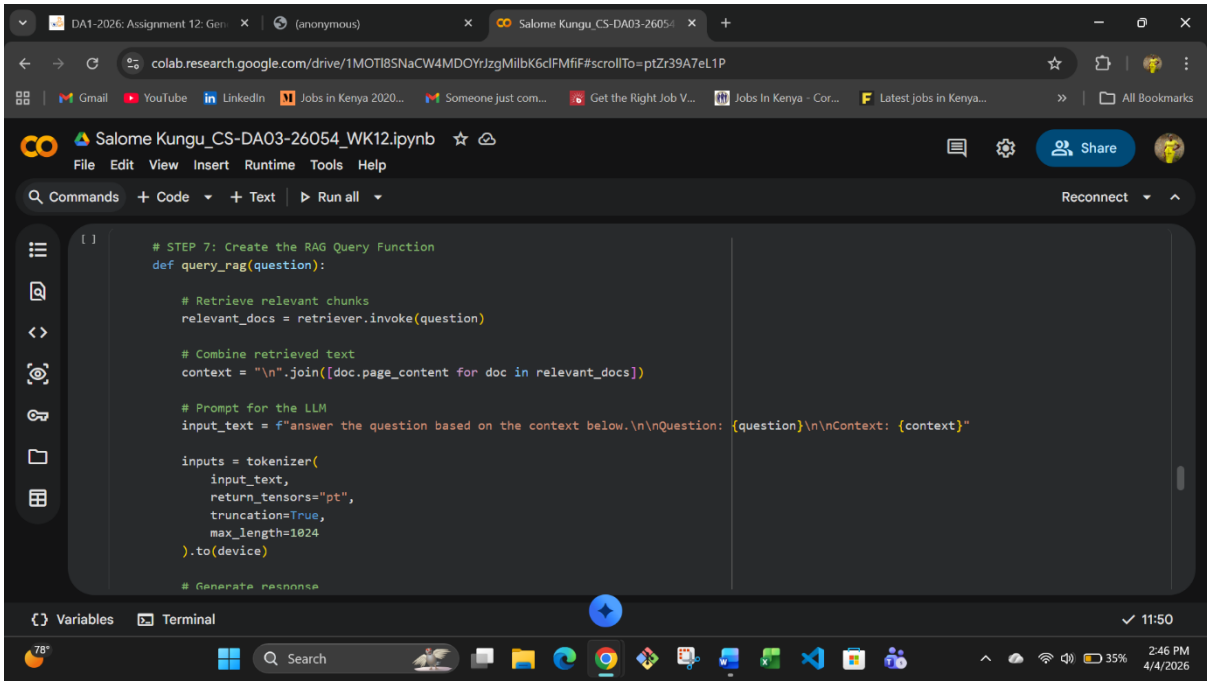
Task 7: Implement the RAG Query Function

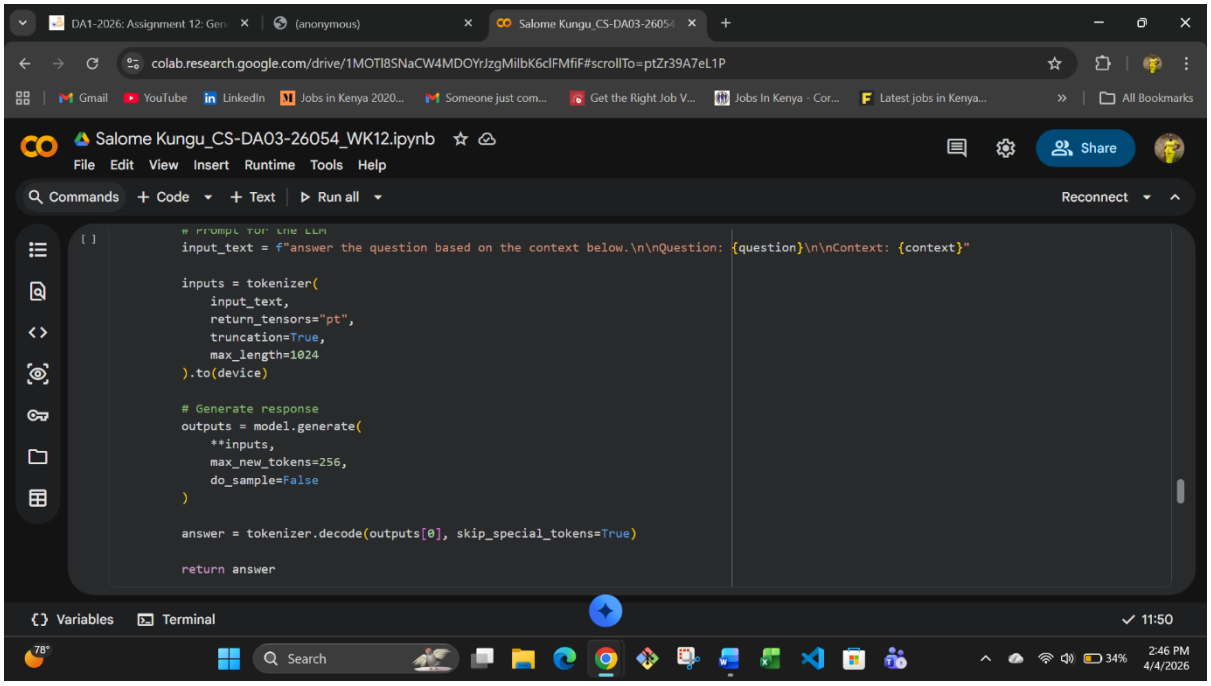
What

- Created a function that retrieves relevant document chunks from the FAISS database and sends them with the user query to the language model.

Why

- This step forms the **core of the Retrieval-Augmented Generation pipeline**, ensuring that answers are grounded in retrieved document context.





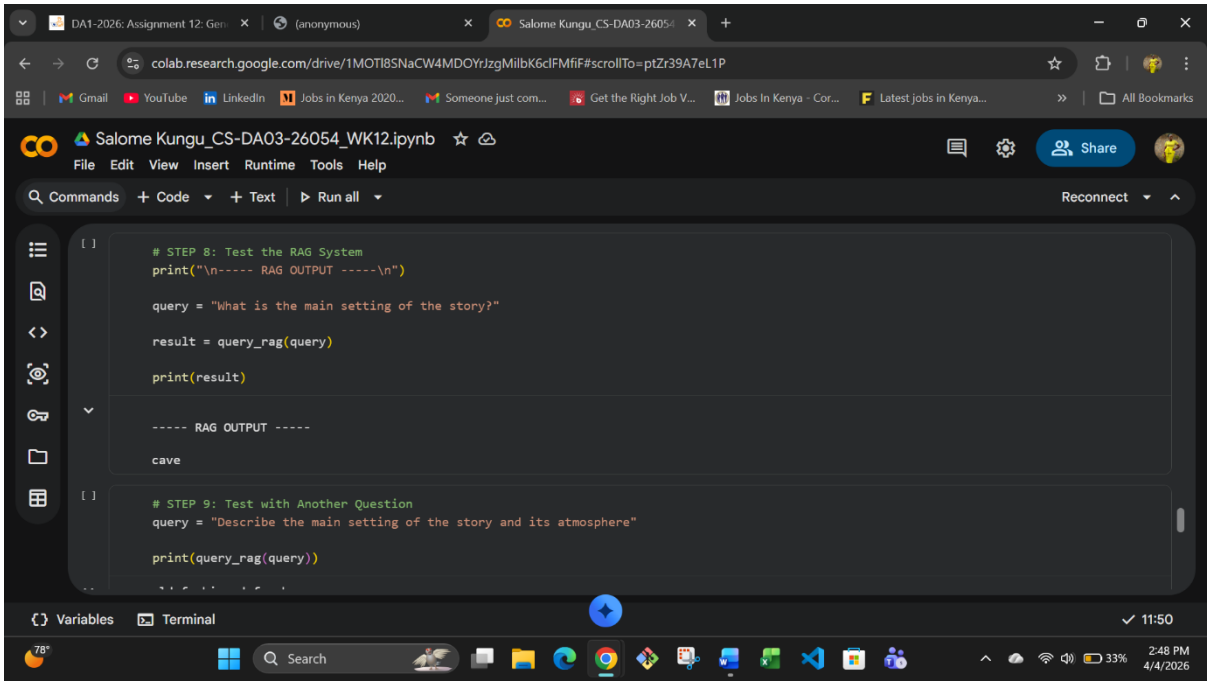
Task 8: Execute the RAG Query

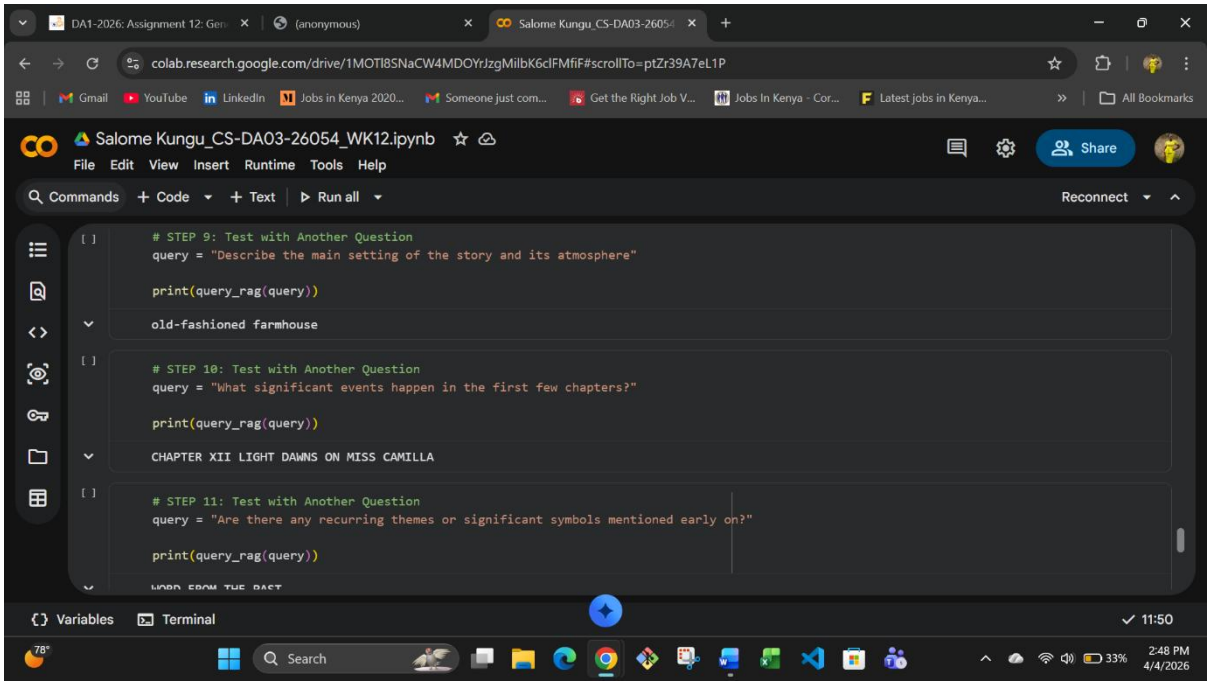
What

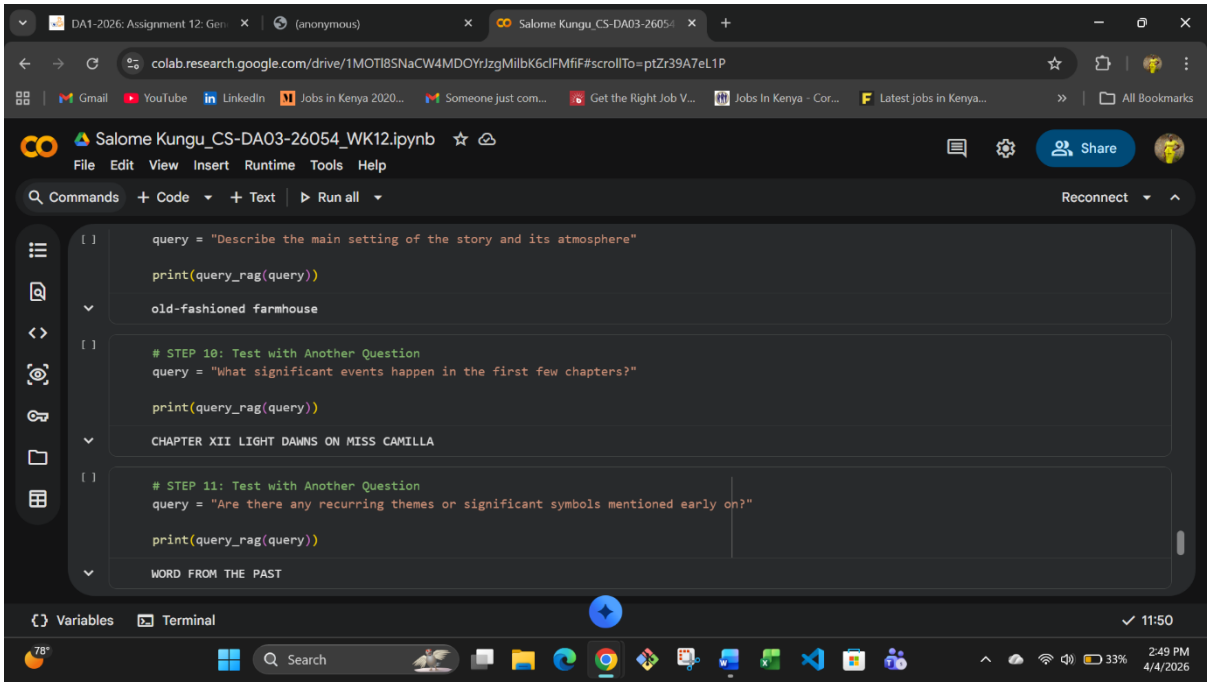
- Executed a query asking the system to summarize key information, skills, and experience from the document.

Why

- This step demonstrates how the RAG pipeline retrieves relevant document context and generates meaningful responses using the language model.







Conclusion

This assignment demonstrated how **Retrieval-Augmented Generation (RAG)** combines information retrieval and generative AI to improve question-answering systems.

By splitting documents into chunks, generating embeddings, storing them in a FAISS vector database, and retrieving relevant information before generating responses, the system produces more accurate and context-aware answers.

Through this assignment, practical experience was gained in working with **LangChain, Hugging Face Transformers, FAISS vector databases, and prompt engineering techniques**, which are widely used in modern AI-powered applications.